IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Visión artificial: ver, reutilizar y localizar

CNN, transfer learning, detección y segmentación · un recorrido por S11_01 a S11_03

Sprint 11 · recorrido global de los cuadernos S11_01 a S11_03

DÓNDE ESTAMOS

De las tablas a las imágenes

Hasta S10: tablas

Cada fila es una reserva, cada columna un dato.

El **MLP** recibe una lista de números.

S11: imágenes

Una imagen es una **tabla de números** (píxeles).

Una **CNN** mira **zonas pequeñas** con filtros y respeta la forma de la imagen.

Tres preguntas

¿**Qué** hay? (clasificar)

¿**Qué y dónde**? (detectar)

¿**Qué píxeles**? (segmentar)

Con imágenes, el reto no es solo aprender: es aprender con pocas fotos y medir bien dónde está cada cosa.

S11 usa dígitos de 8 × 8, dibujos de alojamientos y escenas de playa de TurisData.

LOS TEMAS DE HOY

Tres cuadernos, tres preguntas

Tema	La pregunta que responde	Cuaderno
CNN	¿Cómo **ve** una red una imagen?	S11_01
Transfer learning	¿Cómo **reutilizo** una red si tengo pocas fotos?	S11_02
Detección y segmentación	¿**Dónde** está cada cosa y **qué píxeles** ocupa?	S11_03

Un hilo común: **medir con honestidad**. Una cifra solo vale si sabes **con qué se compara**.

Recorrido: 1 · CNN → 2 · Transfer learning → 3 · Detección y segmentación.

Una imagen es una tabla de números

0	0	0	2	16	16	2	0
0	0	0	4	16	16	2	0
0	1	4	12	16	12	0	0
0	7	16	16	16	12	0	0
0	0	3	10	16	14	0	0
0	0	0	8	16	12	0	0
0	0	0	6	16	16	2	0
0	0	0	2	12	15	4	0

Un dígito real de 8 × 8 píxeles: 0 = blanco, 16 = tinta

Escala de grises

Una tabla **(alto, ancho)**: un dígito de 8 × 8 son **64 números**.

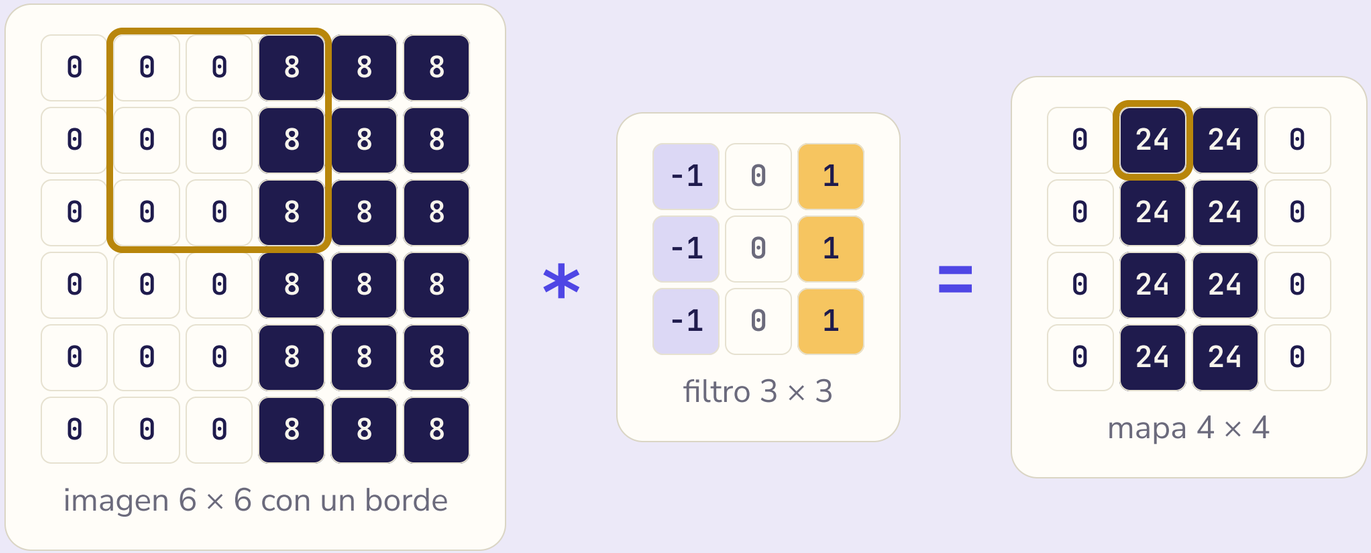
Color: 3 canales

Una tabla por color (rojo, verde, azul): **(alto, ancho, 3)**. Una foto de 32 × 32 son **3 072** números.

Un lote de imágenes

Añade una dimensión delante: **(n, alto, ancho, 3)**.

La convolución: un filtro que se desliza



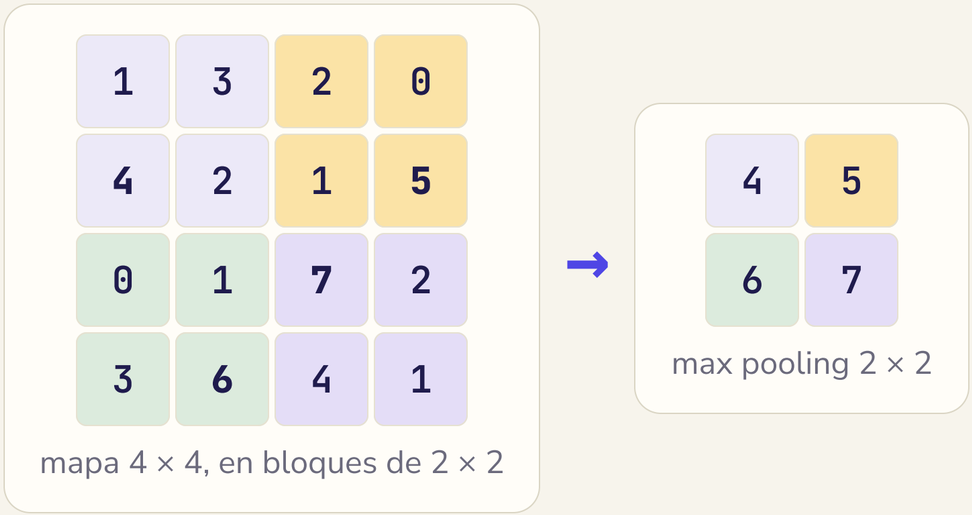
Una posición, un número

En el marco: cada fila da 0·(−1) + 0·0 + 8·1 = 8; las tres suman **24**. El filtro se desliza y repite la cuenta.

El mapa dice dónde está el patrón

Valor alto (24): hay un **borde vertical**. Valor 0: no. La CNN **aprende sus propios filtros**.

Pooling: quedarse con lo más fuerte



El máximo de cada bloque

El mapa se queda en la **mitad** de lado: menos cálculo.

Tolera pequeños desplazamientos

Si el 9 cambia de casilla **dentro del bloque**, el máximo sigue siendo 9.

Una CNN: filtros, pooling y una decisión

Capa (ejemplo 12 × 12, 5 clases)	Salida	Parámetros
Entrada: imagen en grises	12 × 12 × 1	0
Conv2D · 8 filtros de 3 × 3 (same)	12 × 12 × 8	80
MaxPooling2D · 2 × 2	6 × 6 × 8	0
Conv2D · 16 filtros de 3 × 3	6 × 6 × 16	1 168
MaxPooling2D · 2 × 2	3 × 3 × 16	0
Flatten (aplanar)	144	0
Dense · 5 clases, softmax	5	725
Total		**1 973**

De trazos a formas

Las primeras capas detectan **trazos simples**; las siguientes los combinan en **formas** cada vez más complejas.

Pocos parámetros

El **mismo filtro** se reutiliza en toda la imagen.

El patrón: [Conv → ReLU → Pool] × n → Flatten → Dense.

Antes de entrenar: forma, escala y reparto

Qué tengo	Forma	Números
Imagen en grises 10 × 10	(10, 10)	100
La misma, con canal explícito (Keras)	(10, 10, 1)	100
Foto en color 32 × 32	(32, 32, 3)	3 072
Un lote de 20 fotos en color	(20, 32, 32, 3)	61 440

1 · Normalizar

Se divide entre el máximo (16 en los dígitos, 255 en una foto): quedan entre **0 y 1**.

2 · Entrenamiento y prueba

train_test_split con **stratify**: 2 000 imágenes → **1 500 y 500**, con la misma proporción de cada clase.

Si Keras protesta por la forma, mira el canal: falta una dimensión al final.

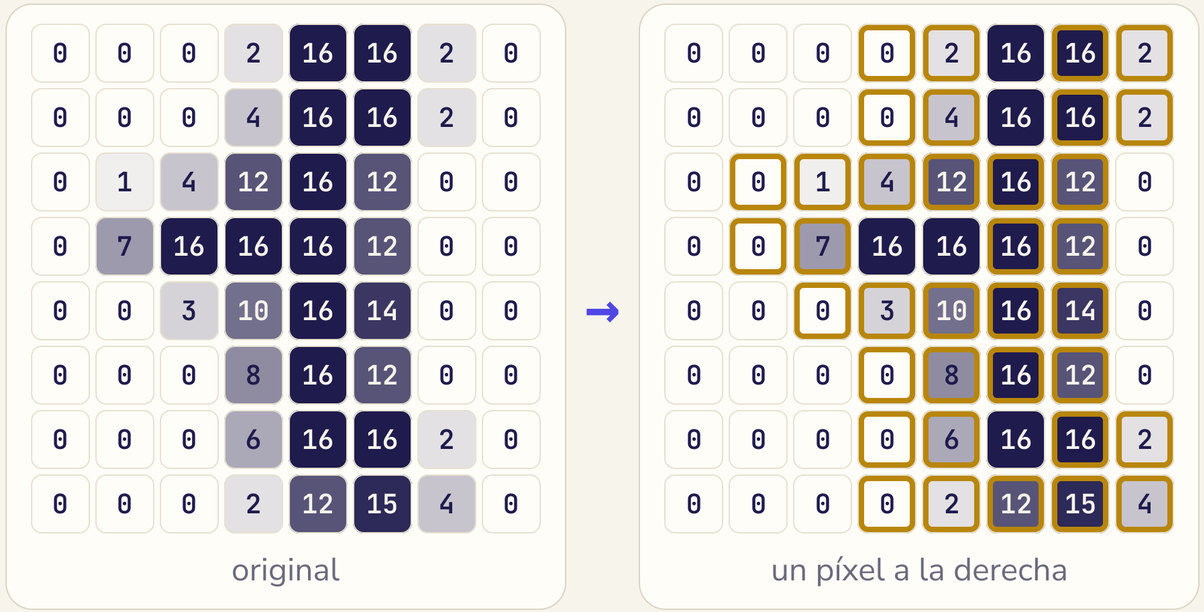
¿Cuánto mide la salida de una capa?

n	k	p	s	Cuenta	Salida
10	5	0	1	(10 – 5 + 0) ÷ 1 + 1	**6**
10	3	0	2	(10 – 3 + 0) ÷ 2 + 1	**4**
9	3	1	2	(9 – 3 + 2) ÷ 2 + 1	**5**
28	5	2	1	(28 – 5 + 4) ÷ 1 + 1	**28**

salida = (n – k + 2·p) ÷ s + 1 (n entrada · k filtro · p relleno · s paso; ÷ es división entera)

Con paso 1 y relleno justo (**padding="same"**), la salida mide **lo mismo que la entrada**: última fila.

Si la imagen se mueve un píxel…



Cambian 36 de 64 casillas

Para un **MLP**, cada casilla es una entrada distinta: el dígito movido le parece **otro dato**.

Aumento de datos

Variaciones al azar **solo al entrenar**: **RandomTranslation**, **RandomRotation**, **RandomZoom**.

Con dígitos desplazados: MLP 0,144, CNN 0,264, CNN con aumento 0,651. Sin desplazar, el aumento cuesta: 0,898 frente a 0,976.

CNN frente a MLP en dígitos 8 × 8

Exactitud en prueba (450 dígitos)



La CNN gana por poco: en imágenes pequeñas y limpias, **ambas funcionan**.

La ventaja crece con la foto

La CNN **comparte los filtros** por toda la imagen: sus parámetros casi no cambian con el tamaño.

Primera capa según la foto	Valores de entrada	Dense(64)	Conv2D(16 filtros 3 × 3)
8 × 8 en grises	64	4 160	160
32 × 32 en color	3 072	196 672	448
96 × 96 en color	27 648	1 769 536	448

Pocas fotos etiquetadas: no empieces de cero



hotel

pocas fotos



apartamentos

pocas fotos



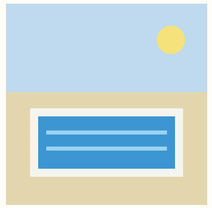
casa rural

pocas fotos



camping

pocas fotos



piscina

pocas fotos

Desde cero

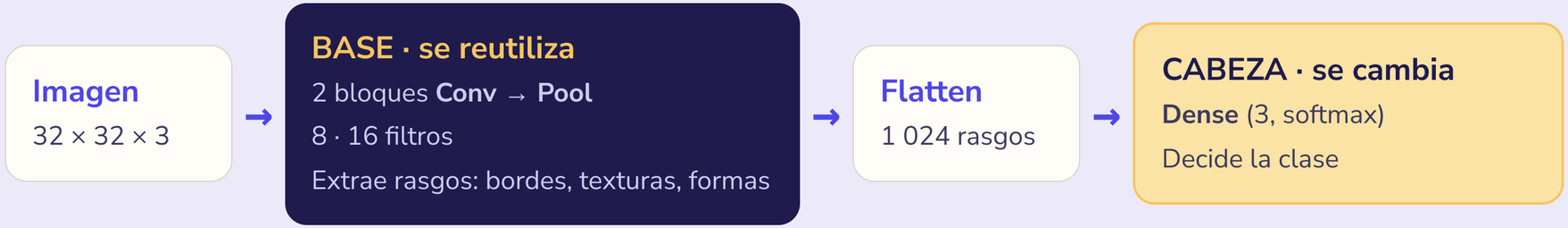
La red tiene que aprender a **mirar** y a **decidir** solo con tus pocas fotos: tiende a **memorizar**.

Reutilizando

Se parte de una red que **ya sabe mirar** y solo se le enseña lo nuevo: hacen falta **menos fotos**.

Bordes, texturas y formas sirven para casi cualquier imagen: se pueden reutilizar.

Una red tiene base y cabeza



1 392
parámetros de la base (red de juguete)

3 075
parámetros de la cabeza (3 clases)

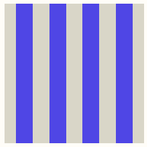
4 467
parámetros en total

La **base** aprende a mirar; la **cabeza** aprende qué hay en **tu problema**.

Preentrenar la base en una tarea parecida



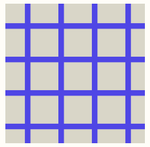
rayas horizontales



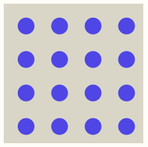
rayas verticales



diagonales



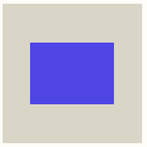
cuadrícula



puntos



círculo



rectángulo



triángulo

En la vida real: ImageNet

Se descarga una base **ya entrenada** con millones de fotos de objetos reales.

En el cuaderno, sin internet

La base aprende antes **8 clases de formas y texturas** (3 200 imágenes) y llega a **0,938** de exactitud.

Bordes, esquinas y patrones repetidos **también sirven** para reconocer edificios y tiendas de campaña.

Congelar la base y entrenar solo la cabeza

BASE · congelada
trainable = False
sus pesos no cambian

→

CABEZA nueva · se entrena
una capa Dense para tus 5 clases

Estrategia (30 imágenes)	Parámetros que se entrenan	Exactitud en prueba
CNN desde cero	35 109 (todos)	0,800
Transfer: base congelada	11 525 (solo la cabeza)	0,833

Se congela **una copia** de la base, no la original. Con un solo entrenamiento la diferencia es pequeña: se comprueba con **varias semillas**.

Fine-tuning: ajustar el último bloque

Fase	Bloque 1	Bloque 2	Bloque 3	Cabeza	Tasa
1 · Solo la cabeza	congelado	congelado	congelado	se entrena	0,001
2 · Fine-tuning	congelado	congelado	**se entrena**	se entrena	**0,0001**

Regla de oro

Primero la cabeza, **después** el fine-tuning, con una tasa **10 veces menor**: se ajustan los rasgos específicos sin destrozar los generales.

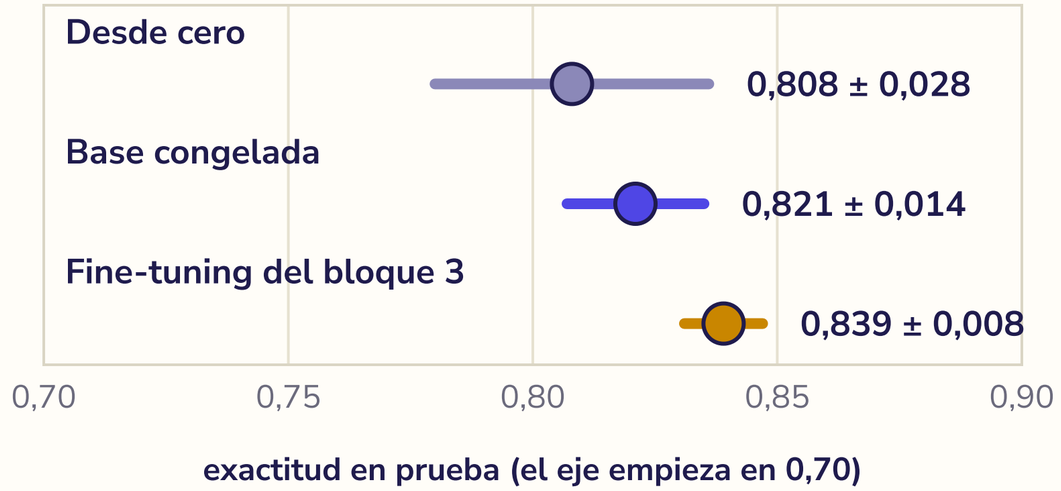
Error típico

Cambiar **trainable** y **no recompilar**: el cambio no surte efecto. Usa **compile** otra vez.

Con muy pocas imágenes un entrenamiento suelto es muy variable: se repite con **varias semillas**.

Tres estrategias, tres semillas

Media ± desviación de 3 semillas · 30 imágenes de entrenamiento



Cómo leerlo

Con pocas imágenes, la base preentrenada ayuda: **0,808** → **0,839** con fine-tuning.

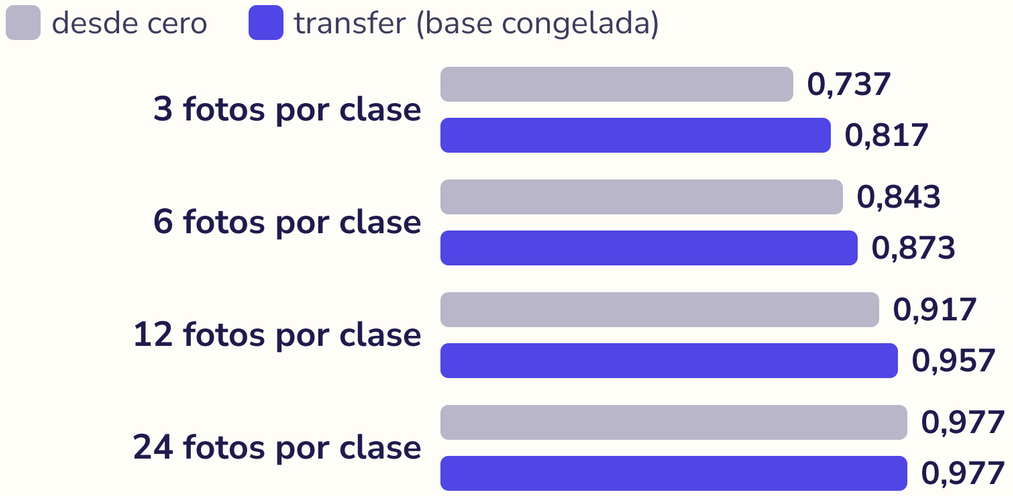
Pero la mejora es **modesta** y los rangos se **solapan**.

Qué aprender

El **método** importa más que la cifra. No se promete que el transfer learning gane **siempre**.

¿Cuántas fotos hay que etiquetar?

Exactitud en prueba según las fotos de cada clase



Gana con pocas fotos

Con **3, 6 y 12** por clase, el transfer **supera** a empezar de cero.

Con muchas, empatan

Con **24** por clase, la ventaja **desaparece** (0,977 las dos).

MobileNetV2: la misma estructura

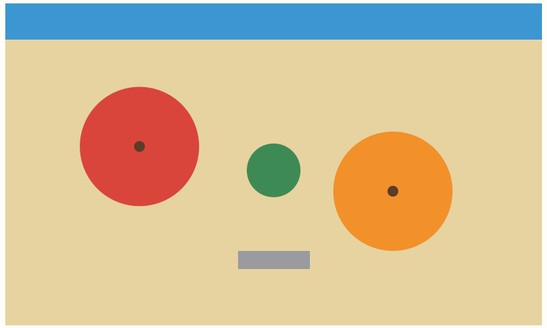
Qué cambia	Con MobileNetV2 e ImageNet
Base	Ya entrenada con **millones de fotos reales**
Preprocesado	El que pide la red (**preprocess_input**)
Tamaño de entrada	Se **reescalan** las fotos a lo que pide la red
Batch norm	Al congelar la base: **training=False**

La estructura es la misma: base congelada + cabeza nueva (solo 6 405 parámetros entrenables en el ejemplo).

Sin internet, la celda usa **pesos aleatorios**: los de ImageNet se prueban en Colab y **no están verificados** en el taller.

Tres tareas con imágenes

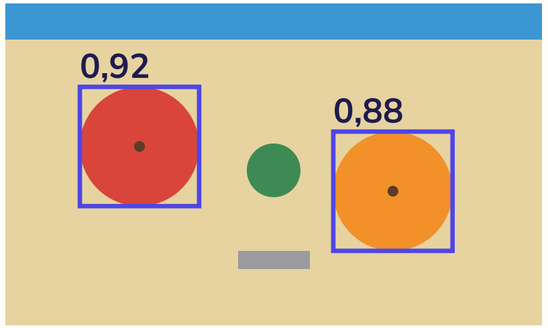
Clasificación · ¿qué hay?



Salida: una etiqueta

«playa»

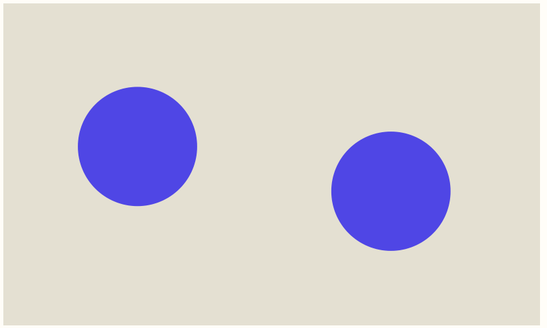
Detección · ¿qué y dónde?



Salida: cajas + clase + confianza

2 × «sombrilla»

Segmentación · ¿qué píxeles?



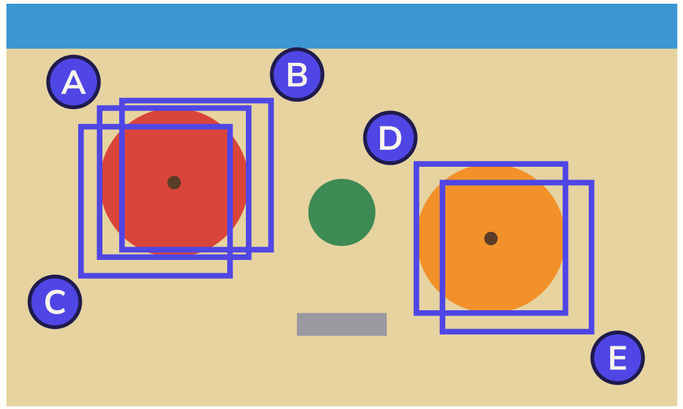
Salida: una máscara

píxeles de sombrilla en azul

TurisData: **contar sombrillas** es detección; **medir la superficie** de la piscina es segmentación.

Cómo trabaja un detector

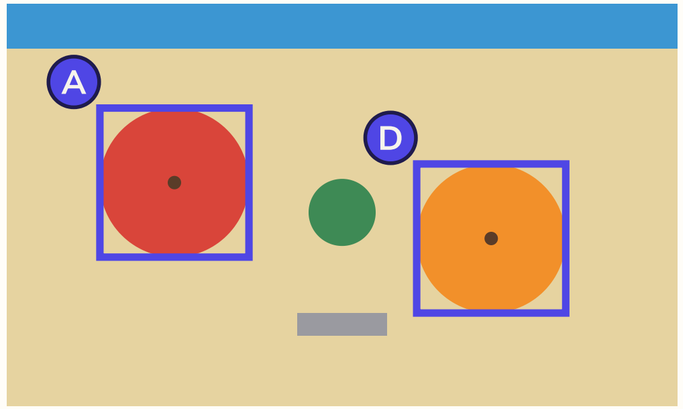
① Proponer: 5 cajas candidatas



cada una con clase y confianza

NMS →

② Limpiar: 2 cajas, una por objeto

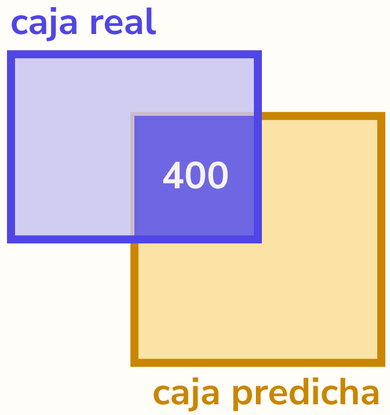


A y D se quedan

③ **Evaluar:** una caja acierta si su **IoU ≥ 0,5** con una caja real; de ahí salen **precisión, recall y AP.**

IoU: cuánto se solapan dos cajas

Intersección ÷ unión



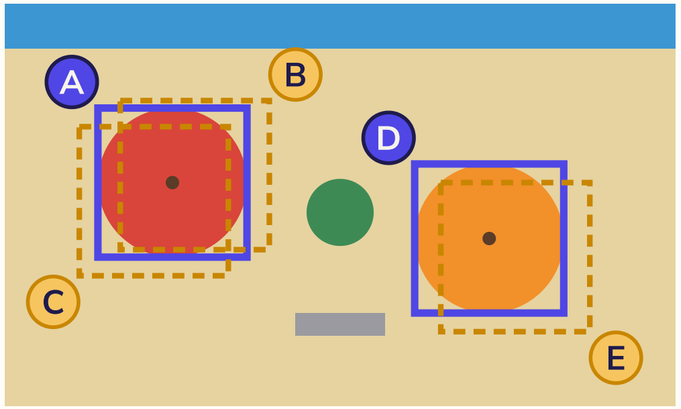
Cajas como [x1, y1, x2, y2]

Áreas: 40 × 30 y 40 × 40	1 200 y 1 600
Intersección: 20 × 20	400
Unión: 1 200 + 1 600 – 400	2 400
IoU = 400 / 2 400	**0,167**

Caja predicha	IoU	¿Acierto (≥ 0,5)?
Casi encima	0,724	sí
La del dibujo	0,167	no
Lejos	0	no

NMS: una caja por objeto

Las cajas del detector



trazo continuo = se queda · discontinuo = se descarta

Orden por confianza: **A** 0,92 · **D** 0,88 · **B** 0,85 · **C** 0,60 · **E** 0,55

Paso	Se queda	Se descarta (IoU > 0,5)
1	A (0,92)	B (0,677) · C (0,620)
2	D (0,88)	E (0,565)
Resultado	A y D	una caja por sombrilla

Ojo con el **umbral**: dos sombrillas vecinas con IoU 0,25 se perderían con un umbral de 0,2.

Precisión, recall y AP

Puesto	Confianza	¿Acierto?	Aciertos	Precisión	Recall
1	0,97	sí	1	1,00	0,20
2	0,91	no	1	0,50	0,20
3	0,84	sí	2	0,67	0,40
4	0,78	sí	3	0,75	0,60
5	0,66	no	3	0,60	0,60
6	0,52	sí	4	0,67	0,80

0,667

precisión: 4 de 6, de lo que **dice**, qué parte es cierta

0,80

recall: 4 de 5, de lo que **hay**, cuánto encuentra

0,633

AP: área bajo la curva precisión–recall

5 sombrillas reales y 6 cajas ordenadas por confianza. **AP = 0,2 · (1 + 0,75 + 0,75 + 2/3)**.

Segmentar: medir máscaras con IoU y Dice



IoU de máscaras

Píxeles de la **intersección** entre píxeles de la **unión**: 6 / 12 = **0,50**.

Dice

Dos veces la intersección entre la **suma** de las dos áreas: 2·6 / (8 + 10) = **0,667**.

El Dice da más peso a la intersección: **vale igual o más que la IoU**. Muy usado en medicina.

Un detector de color, sin entrenar nada

Detector de color · 30 escenas, 171 sombrillas reales



Acierta **94 de las 106** cajas, pero solo encuentra **94 de las 171** sombrillas.

AP según el umbral de IoU



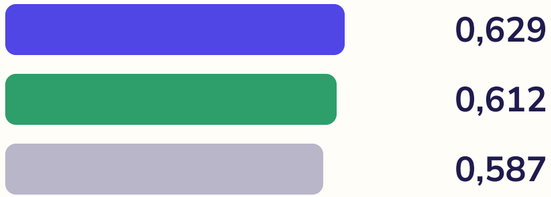
mAP (media) = **0,36**

Encuentra sombrillas, pero sus cajas no son precisas: con IoU 0,9 el AP cae a 0.

Se le escapan las sombrillas **pastel** y las **solapadas**.

Un método clásico bien pensado compite

IoU media de la máscara



Dice medio del umbral de color: 0,756.

Sin entrenar nada

La máscara del detector de color, más una limpieza, **ya es una segmentación**.

Mini red encoder-decoder

Reduce la imagen con convoluciones y la **agranda** hasta dar una probabilidad por píxel.

Casi iguales: en un problema limpio, un método clásico bien pensado es un buen punto de partida.

YOLO: el detector moderno

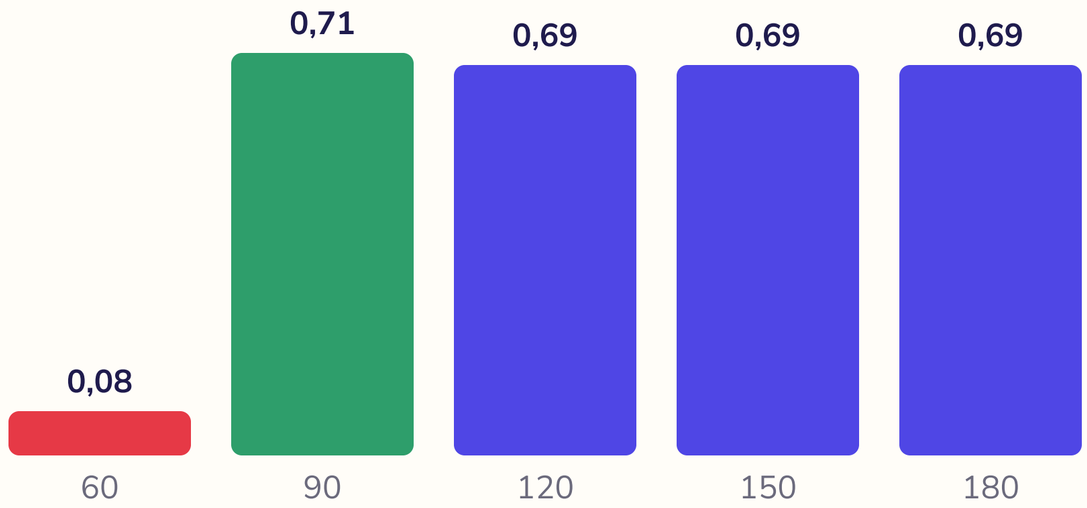
	Detector clásico (OpenCV)	Detector moderno (YOLO)
Qué aprende	Nada: reglas hechas a mano (color, plantilla)	Todo el detector, con miles de imágenes con cajas
Cajas por imagen	Las de la regla; el NMS lo aplicas tú	Muchas en una pasada; ya aplica NMS por dentro
Va mejor cuando	El objeto tiene un color inconfundible	Hay sombras, oclusiones o fondos complejos
Se evalúa con	AP con IoU ≥ 0,5	mAP@[.5:.95]

Lo que programaste a mano (IoU, NMS y AP) es justo lo que ocurre dentro y alrededor de YOLO.

La celda de YOLO necesita internet y **no está verificada** en el taller: se prueba en Colab.

Contador de sombrillas: ajustar un umbral

F1 según la saturación mínima (sat_min)



F1 mezcla precisión y recall en un solo número.

Muy permisivo (60)

La **arena** «cuela» como sombrilla: precisión **0,07**, F1 **0,08**.

El mejor (90)

F1 **0,71**. Más exigente (120–180) apenas baja: **0,69**.

Un parámetro mal puesto puede **hundir** un detector: se **mide** con precisión, recall y F1.

QUÉ TÉCNICA USO

Según lo que necesites de la imagen

Necesito…	Técnica	Ojo con…
Saber **qué hay** en la foto	**CNN** (clasificar)	Normalizar y dar forma **(alto, ancho, canal)**
Clasificar con **pocas fotos**	**Transfer learning**: base congelada + cabeza	Repetir con varias **semillas**; fine-tuning con tasa pequeña
Saber **qué y dónde**	**Detección**: cajas + NMS	**IoU ≥ 0,5**; medir con precisión, recall y AP
Saber **qué píxeles**	**Segmentación**: máscaras	Medir con **IoU y Dice**
Que aguante **movimientos**	**Aumento de datos**	Solo al entrenar; puede costar exactitud sin mover

Empieza por lo simple (color, base congelada) y complica solo si la medida lo justifica.

MAPA FINAL

Idea, cómo sabes si funciona y la trampa típica

Idea	Cómo sabes si funciona	La trampa típica
CNN	**Exactitud en prueba** frente a un MLP	Creer que la CNN **siempre** gana; olvidar el **canal**
Transfer learning	**Media y desviación** de varias semillas	Fiarse de **un solo** entrenamiento; no **recompilar**
Detección	**IoU**, precisión, recall y **AP**	Un **umbral de NMS** mal elegido; mirar solo la precisión
Segmentación	**IoU y Dice** de las máscaras	Confundir **detectar** con **segmentar**

En visión, una cifra sola no basta: **compara con un punto de partida** y **mira la imagen.**

PARA LLEVARSE

Lo que tienes que poder decir

CNN

Una imagen es una **tabla de números**. **Filtros + pooling + decisión**.

Transfer learning

Base congelada + cabeza nueva. Mide con **varias semillas**.

Detección

Cajas, IoU ≥ 0,5 y NMS. Se mide con **precisión, recall y AP**.

Segmentación

Máscaras de píxeles. Se mide con **IoU y Dice**.

Los detalles, en las presentaciones S11_01 a S11_03 y en los cuadernos.